文件操作

在日常生活中，文件操作主要包括打开、关闭、读、写等操作

操作过程中请务必注意文件路径的书写

- 绝对路径
- 相对路径

1、文件的打开

在Python，使用open函数，可以打开一个已经存在的文件，或者创建一个新文件

- 基本语法：

```
open(name, mode, encoding)
# name：是要打开的目标文件名的字符串(可以包含文件所在的具体路径)。
# mode：设置打开文件的模式(访问模式)：只读、写入、追加等。
# encoding:编码格式（推荐使用UTF-8）
```

- 示例代码：

```
f = open("C:/code/bill.txt", "r", encoding="UTF-8")
# encoding的顺序不是第三位，所以不能用位置参数，用关键字参数直接指定
# f是open函数的文件对象，可以使用对象 .属性或对象 .方法对其进行访问
```

打开模式

- 文件常用的三种基础访问模式，可通过mode指定。
- `r` -> `read` (读取)， `w` -> `write` (写入)， `a` -> `append` (追加)

其他文件访问模式

- `'r'` - 只读模式，文件的指针将会放在文件的开头，默认模式。如果文件不存在则会抛FileNotFoundError。
- `'w'` - 写入模式。如果文件存在则覆盖原有内容；如果文件不存在则创建新文件。
- `'a'` - 追加模式。如果文件存在则在文件末尾追加内容；如果文件不存在则创建新文件。
- `'x'` - 创建模式。如果文件已存在则操作失败。
- `'b'` - 二进制模式。通常与上面的模式之一结合使用，如 `'rb'` 或 `'wb'`。
- `'t'` - 文本模式（默认），也可以与上面的模式之一结合使用，如 `'rt'` 或 `'wt'`。
- `'+'` - 更新模式，即同时具有读和写的功能。比如 `'r+'`、`'w+'` 和 `'a+'`。

2、文件的读取

操作	功能
文件对象.read(num)	读取指定长度字节 不指定num读取文件全部
文件对象.readline()	一次读取一行，包含\n
文件对象.readlines()	读取全部行，返回列表
for line in 文件对象	for循环文件行， 一次循环得到一行数据
文件对象.close()	关闭文件对象
with open() as f	通过with open语法打开文件，可以自动关闭

- 每次读取会从上一次读取结束的位置开始
- 每次open()中的内容只能被读取一次

1、 read方法

语法：文件对象.read([size] ）按指定个数的字符或字节读取

- size表示要从文件中读取的数据的长度（单位是字符/字节）
- 如果没有传入size，那么就表示读取文件中所有的数据。

案例：读取文本文件

```
# 1 打开文件
f = open("a.txt", 'r', encoding='UTF-8')

# 2 读取文件
# 读取3个字符的文本
# c = f.read(3)
# 读取文件所有数据
c = f.read()

# 3 关闭文件
f.close()
```

注意：读取到回车换行时，算一个字符

2、按行读取

如果你想逐行读取文件，可以使用 `readline()` 或 `readlines()` 方法，或者直接遍历文件对象。

（1）readline( ) 一次读取一行内容，会读取换行符\n，如果读取到完最后一行后继续读取，返回 ''（空字符串）

案例2：按行读取文件

```
f = open("a.txt", 'r', encoding='UTF-8')

# 一次读取一行，会读取\n
c = f.readline()
# print(c, end='')  # 使用 end='' 避免额外换行

c = f.readline() # 'helloworld'
c = f.readline() # ''
print(c, end='')
```

（2）readlines( ) 按照行的方式把整个文件中的内容进行一次性读取，并且返回的是一个列表，其中每一行的数据为一个元素。

案例3：readlines使用示例

```
f = open("a.txt", 'r', encoding='UTF-8')

# 读取所有行，存入list中
c = f.readlines()
print(c)   # ['你好世界\n', 'helloworld']
```

（3）for循环读取，语法：

```
for line in 文件对象
    print(line)
```

案例4：for循环读取每一行数据

```
# 每一个line临时变量，就记录了文件的一行数据
for line in open("C:/code/test.txt", "r", encoding="UTF-8"):
    print(line)
```

3、关闭文件

（1）如果不调用close，同时程序没有停止运行，那么这个文件将一直被Python程序占用，无法操作

```
f = open("a.txt", "r", encoding="UTF-8")
# 需要执行代码
f.close()
```

- 代码中不关闭文件对象，且python程序未停止运行，无法对文件删除重命名等操作：

（2）自动close

通过在with open的语句块中对文件进行操作,可以在操作完成后**自动关闭**close文件。即使出现异常也会自动调用关闭文件操作。

语法： `with open() as f`

案例5：使用 with open 打开并自动关闭文件

```
with open("a.txt", "r", encoding="UTF-8") as file:
  file.readlines()

with open('a.txt', 'r', encoding='utf-8') as file:
    for line in file:
        print(line, end='')  # 使用 end='' 避免额外换行
```

3、文件的写入

1、覆盖式写入

案例1：覆盖式写入文本

```
# 文件如果不存在，使用"w"模式，会创建新文件
# 文件如果存在，使用"w"模式，会将原有内容清空
f = open("b.txt", 'w', encoding='UTF-8')

f.write('你好世界\n')
f.write('helloworld')

f.flush()
f.close()
```

- 直接调用 `write` ，内容并**未真正**写入文件，而是会积攒在程序的内存中，称之为缓冲区
- 当调用flush的时候，内容会真正写入文件
  - close()方法，附带flush()方法的功能
- 这样做是避免频繁的操作硬盘，导致效率下降（攒一堆，一次性写磁盘）

2、追加式写入

- 使用 `w` 模式，每次写入会将原有内容清空，写入新内容
- 使用 `a` 模式，文件不存在会创建文件,文件存在会在最后追加内容写入文件

```
f = open("b.txt",  'a', encoding='UTF-8')
# 写入内容
f.write('你好世界\n')
f.write('helloworld\n')
# 刷新缓冲区，一次性写入磁盘
f.flush()
f.close()
```

4、综合练习

```
"""
文件复制
"""
# 需求: 把桌面上的a.txt文件复制到工程中，保存为a2.txt
# 1 打开文件
f1 = open(r'C:\Users\alienware\Desktop\a.txt',  'r', encoding='UTF-8')
f2 = open('a2.txt',  'a', encoding='UTF-8')

# 2 读取f1，并写入到f2
for line in f1:
    f2.write(line)
# 一次性写入
f2.flush()

# 3 关闭文件
f1.close()
f2.close()
```